

Project 1. Transformer

用约 300 行 PyTorch 从零写一个 Transformer encoder，在 ChnSentiCorp 中文情感二分类上做到 dev 准确率 ≥ 0.80 （参考基线约 0.85），并能用注意力热图说清楚模型「在看什么」。

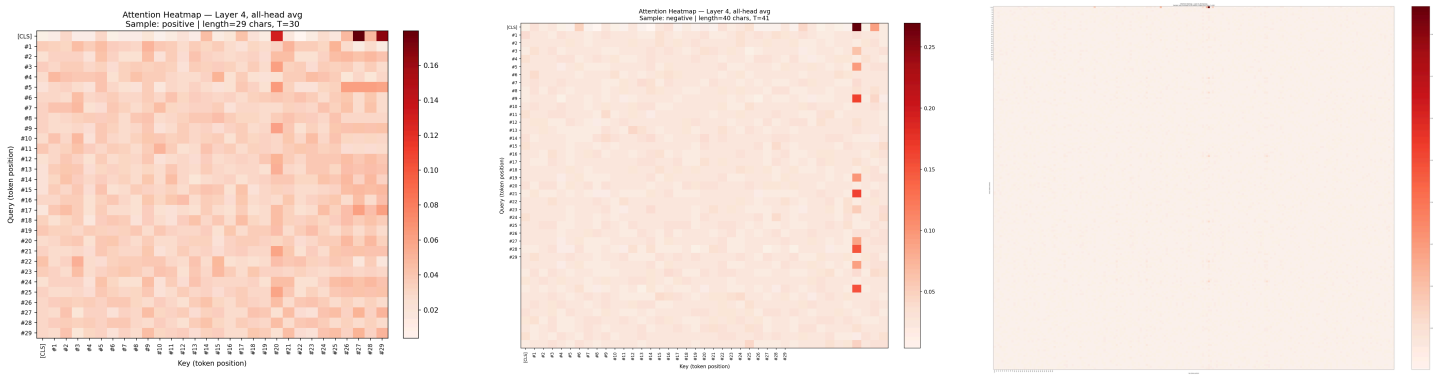
输入 / 输出

	内容
给你	ChnSentiCorp 数据（HF 自动拉，约 9.6K 训练样本，二分类） / PyTorch 2.0+ / 单卡 8GB GPU（CPU 也能跑但慢 $\sim 10\times$ ）
交付	1.ckpt/ best.pt （训练好的模型，预期 5–50 MB） 2. figures/ 下 ≥ 3 张注意力热图 3. eval/result.json（自检结果） 4. 一段 200–500 字的实验观察文字

M1-M5:

具体实现见代码仓库

baseline产生的3张热力图



文件	类型	真实标签	原文
heatmap_1_positive	正面样本	1（好评）	很好的一家酒店哦，以后还会去呢！交通方便，服务好，不错不错
heatmap_2_negative	负面样本	0（差评）	散热一般，从触摸板到右手掌托有些温热，USB只有三个，太少，不过分配到的比较合理
heatmap_3_long	长句	0（差评）	一条 924 字的酒店吐槽（结婚证、马桶坏了等）；画图时 截到前 200 个 token

1. 正面（酒店好评） — 最「像样」的一张

[CLS] 那一行（用来分类的全局表示）在 #20、#27、#29 附近颜色最深。

对照原文（ [CLS] 后 #i = 第 i 个字）：

位置	字	含义
#20	便	「交通方便」
#27、#29	错	「不错不错」

模型做正面分类时， [CLS] 明显在看「方便」「不错」这类好评词

2. 负面（笔记本散热） — 注意末尾

整列最亮在 Key #38 附近，对应句末「比较合理」里的「较」。

这句本身是褒贬夹杂：

- 负面：散热一般、温热、USB 太少
- 末尾：「不过分配比较合理」

图说明： [CLS] 不只看「太少」，还在看句尾转折；对 0/1 二分类来说，模型可能把整句折中信息都收进 [CLS]。情感词不总在句首，attention 会扫到句尾。

3. 长句（酒店投诉） — 稀疏、截断

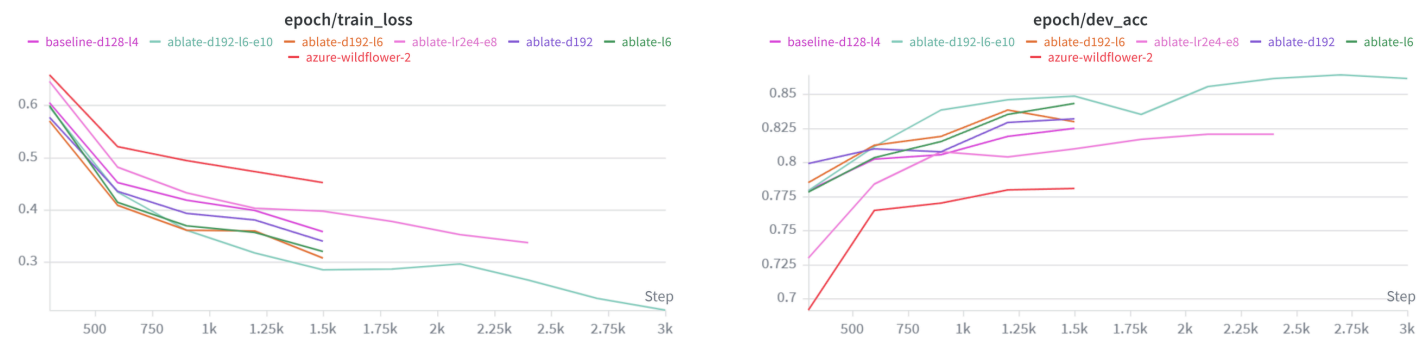
矩阵大、整体很淡，少数竖条略亮，是长序列 + 多头平均后的常见样子：没有单一「情感词」特别突出，注意力更分散。

而且只画了前 200 字（标题里应有 truncated to 200 tokens），后面「马桶坏了」等关键吐槽可能不在图里。长句图更适合说明长度与截断的影响，不宜硬解读某一个词。

超参数调节

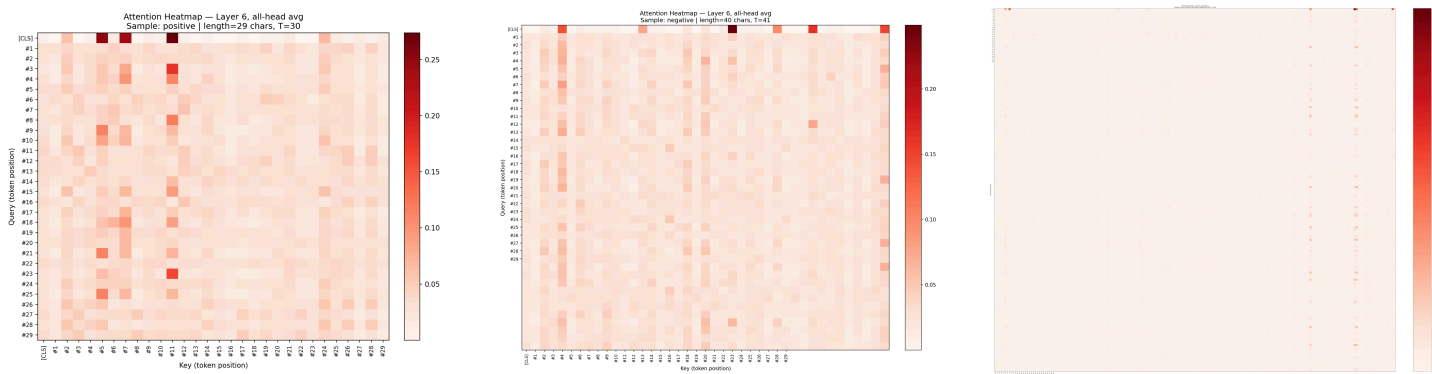
	d_model	n_heads	n_layers	lr	batch_size	epochs
默认	128	4	4	3e-4	32	5
低学习率	128	4	4	1e-4	32	5
加深	128	4	6	3e-4	32	5
加宽	192	4	4	3e-4	32	5
低学习率+增加训练步数	128	4	4	2e-4	32	8

加深+加宽	192	4	6	3e-4	32	5
加深+加宽+增加训练步数	192	4	6	3e-4	32	10



可以看到，加深和加宽网络是最有效的。我们同时组合加深和加宽，并且最后再额外测试增加训练步数。

最好的超参数设置得到的3张热力图：

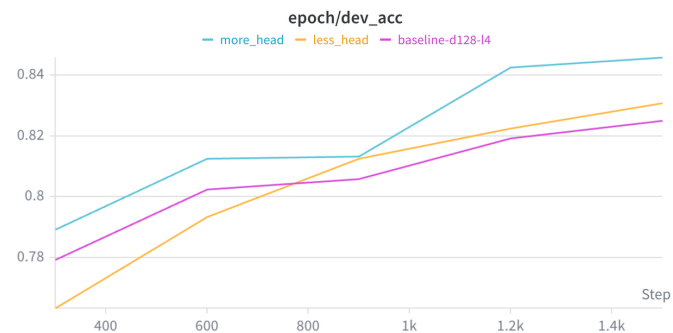


😊 很好的一家酒店哦，以后还会去呢！交通方便，服务好，不错不错
散热一般，从触摸板到右手掌托有些温热，USB只有三个，太少，不过分配到是比较合理
6日晚上9点左右入住……要出示结婚证……无论怎么说……订房的时候从来也没有要求出示结婚证……

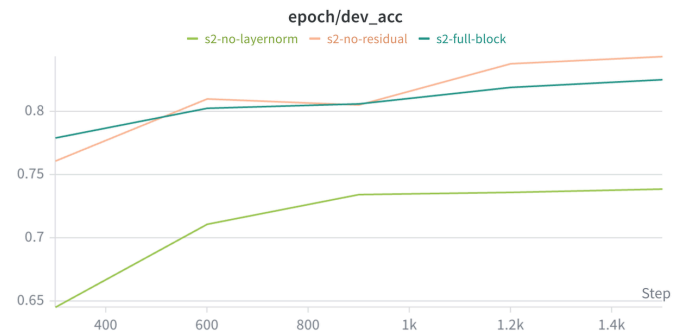
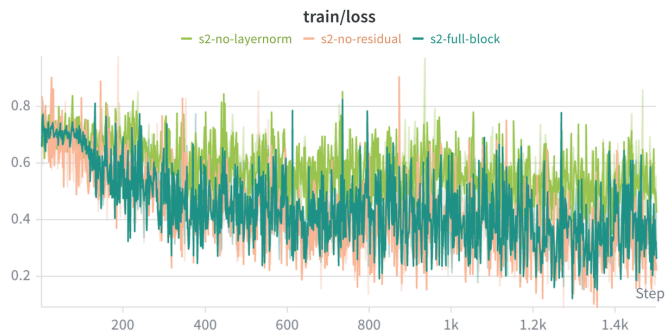
Layer 6 全 head 平均 attention 中，正面样本的 [CLS] 主要关注「好」「酒店」「服务好」等位置，但不像 Layer 4 那样集中在「方便」「不错」；负面样本的 [CLS] 对「只」「太少」「一般」关注最强（#23 最红），较 Layer 4 更能对准抱怨点，句末「合理」仍有响应但非主导。长句样本 attention 较稀疏，[CLS] 多关注标点与「说」「订房」等叙述位置，且仅覆盖前 200 token，后半段关键信息未进入可视化。整体上，更深一层后负面样本的 attention 更可解释，正面与长句则呈现更全局、更分散的模式。

S1: Head消融实验

less_head是2，more_head是8，baseline是4



S2:拆掉Residual或者LayerNorm,记录训练是否收敛



S3:调参使得dev准确率大于0.88

▼ **Config parameters:** {} 27 keys

batch_size: 32

d_ff: 512

d_model: 192

dev_size: 1,200

device: cuda

dropout: 0.1

epochs: 20

label_smoothing: 0

lr: 0.0003

max_len: 200

n_heads: 4

n_layers: 6

no_early_stop: true

no_layernorm: false

no_residual: false

no_wandb: false

num_workers: 2

seed: 42

total_params: 2,890,562

train_size: 9,600

trainable_params: 2,890,562

vocab_size: 4,253

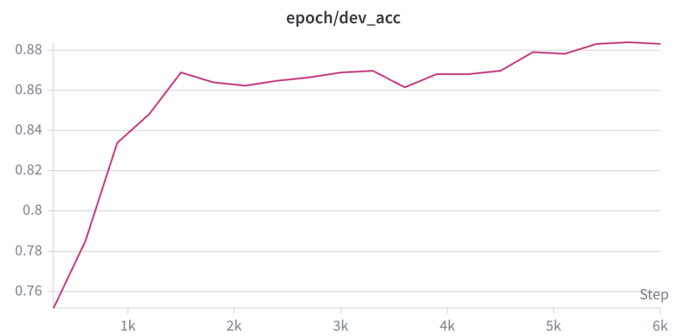
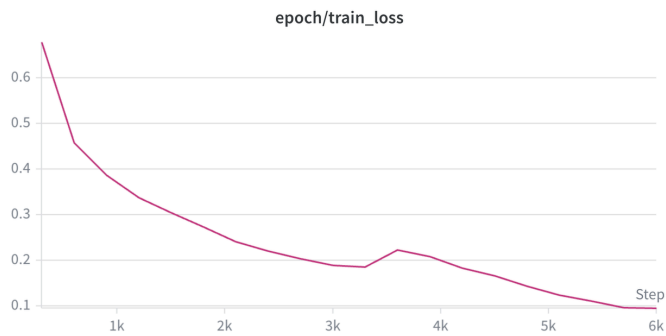
wandb_entity: null

wandb_project: llm-beginner-p1-transformer

wandb_run_name: s3-no-es

warmup_ratio: 0.1

weight_decay: 0.01



S4:绝对PE换成RoPE，与baseline对比dev acc

代码块

```

1  # baseline (绝对 PE)
2  python train.py --d_model 192 --n_layers 6 --epochs 20 --no-early-stop \
3    --pe-type abs --save-best-as s4_abs.pt --wandb-run-name s4-abs-pe
4
5  # RoPE
6  python train.py --d_model 192 --n_layers 6 --epochs 20 --no-early-stop \
7    --pe-type rope --save-best-as s4_rope.pt --wandb-run-name s4-rope

```

